

openclaw

Building & Understanding Personal AI Agents

Prof. Alexander Quispe | Data Science with Python

Pontificia Universidad Católica del Perú | Lecture 15 | 2026

First: which “OpenClaw”?

This lecture

OpenClaw — the AI agent

Open-source autonomous assistant by Peter Steinberger. Runs on your machine, talks through messaging apps, driven by an LLM.

NOT this

OpenClaw — the game engine

A C++ reimplementation of the 1997 platformer *Captain Claw*. Same name, totally unrelated.

If you Google “OpenClaw” you will hit both — we mean the agent.

Learning objectives

By the end of today you will be able to:

1. Explain what an **autonomous LLM agent** is, and how it differs from a chatbot.
2. Describe OpenClaw's **local-first, model-agnostic, channel-based** architecture.
3. Install and run a minimal instance and connect one messaging channel.
4. Build & publish a **Skill**, and decide *Skill vs. MCP server*.
5. Critically assess the **security & privacy risks** of agentic software.
6. Relate these skills to **real data-science workflows**.

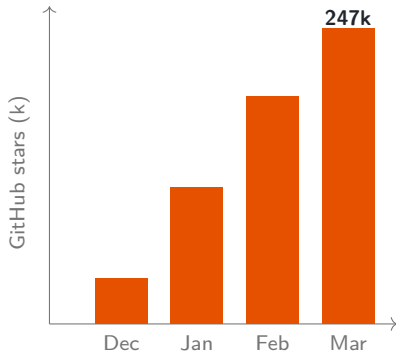
Agenda — 90 minutes

#	Segment	Min
1	Hook: the viral story & what OpenClaw is	10
2	Concepts: agents vs. chatbots; architecture	20
3	Live demo: install → onboard → connect a channel	20
4	Skills & ClawHub; Skills vs. MCP; build one	15
5	Security, ethics & failure cases	15
6	Landscape, your use cases, assignment	10

The Story

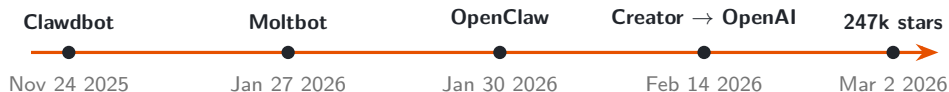
The fastest-growing repo (almost) ever

- Built by **Peter Steinberger**, an Austrian developer.
- Released **Nov 24, 2025** as *Clawdbot*.
- Renamed twice in two months (trademark + taste).
- ~**247,000 GitHub stars** and ~47,700 forks by early March 2026.
- Steinberger **joined OpenAI** (Feb 14, 2026); project continues under a non-profit foundation.



Stars over Dec'25–Mar'26 (illustrative)

A naming saga (a useful timeline)



- *Clawdbot* → *Moltbot* after Anthropic trademark complaints.
- *Moltbot* → *OpenClaw* because the name “never rolled off the tongue.”

What is OpenClaw, in one line?

Definition

A **free, open-source, self-hosted AI agent** that runs on *your* machine and talks to you through the messaging apps you already use.

Channels it speaks on (20+):

WhatsApp ▪ Telegram ▪ Slack ▪ Discord ▪ Signal ▪ iMessage
Microsoft Teams ▪ Matrix ▪ Google Chat ▪ IRC ▪ WeChat ▪ ...

Key shift: from an LLM that *answers* → an LLM that *acts*.

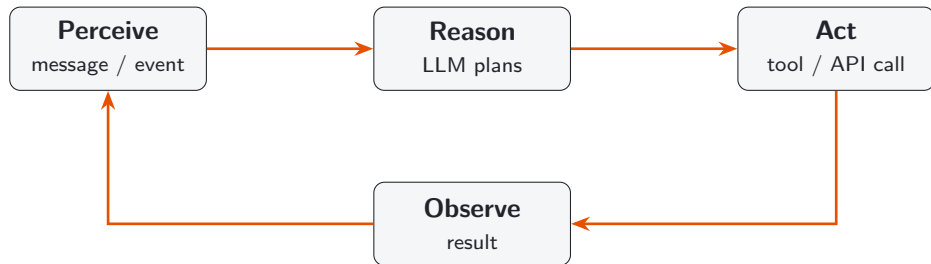
Concepts & Architecture

Chatbot vs. agent

Chatbot	Agent
Returns text	Plans, acts, observes, remembers
Stateless turn	Maintains memory & goals
No side effects	Calls tools / APIs, executes actions
“What is X?”	“Do X, then tell me on Discord”

The agent runs a **loop**, not a single reply.

The agent loop



The LLM is the *brain*; tools are the *hands*; memory is *continuity*.

Four design principles

Local-first

Runs on your hardware; your data stays with you instead of going to a vendor first.

Model-agnostic

Plug in Claude, GPT, Gemini, DeepSeek — not locked to one provider.

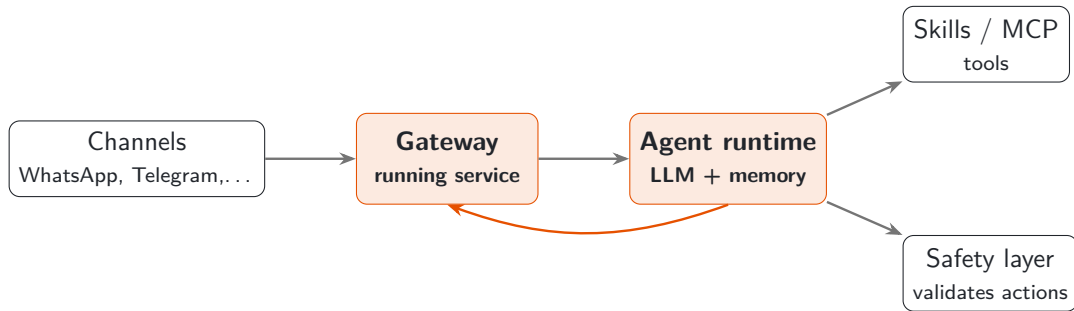
Channel-based UI

No custom app — the interface is a chat thread in apps you already use.

Extensible

Skills (plain-language) + **MCP** servers (standard tool protocol).

Architecture at a glance



Message → Gateway → runtime reasons → calls a tool → safety check → reply.

Tech stack & requirements

Built with

- TypeScript (core) + Swift (Apple clients)
- Local config files (the “local-first” part you can inspect)
- Runs on Windows, macOS, Linux

You need

- **Node.js ≥ 22**
- An API key: Anthropic / OpenAI / Google
- A messaging account to connect (use a **test** one!)

Connects back to Lecture 11 (ModernBERT) & Lecture 13 (fine-tuning): the “brain” is the same kind of model you have already used.

Live Demo

Install (one of):

```
npx clawdbot@latest  
# or  
curl -fsSL https://openclaw.ai/install.sh | bash
```

Then:

1. Run onboarding → paste an API key → pick a model.
2. Connect *one* channel (Telegram bot token or Discord = easiest live).
3. Send the first message; watch it reason and reply.
4. Open the config files on disk — see “local-first” for real.

Demo safety rules (do this every time)

- Run it in a **VM / container**, not your daily machine.
- Use a **dedicated API key with a spend cap**.
- Use a **throwaway** Telegram/Discord account.
- **Never** grant full-disk or terminal access where real secrets live.

Why so cautious?

An agent that can run commands and read files turns a small bug into a big breach. We will see exactly how in the security segment.

Skills & ClawHub

Skills = apps for your agent

- A **Skill** = a SKILL.md (plain-language instructions) + supporting files.
- **ClawHub** = the public registry — **13,700+** community skills (late Feb 2026).
- Mental model: *OpenClaw is the OS; skills are the apps you install.*

dev tools

productivity

smart home

messaging

...

Skill vs. MCP server — when to use which

Skill	MCP server
Plain-language instructions	Separate process exposing tools
Read by the agent at runtime	Standard protocol (MCP)
OpenClaw-specific	Portable: Claude Desktop, Cursor, VS Code...
Fastest for one automation	Best for state / many tool endpoints

Best of both: build an *MCP* server for the capability, wrap it in a *Skill* for the workflow.

Building & publishing a Skill

A minimal SKILL.md:

```
---
```

```
name: daily-arxiv-digest
```

```
description: Summarize new arXiv papers on a topic each morning
```

```
---
```

When asked for my digest:

1. Fetch today's arXiv listing for the given category.
2. Summarize the 5 most relevant titles in 2 lines each.
3. Send the summary to me on Telegram.

Publish: fork github.com/openclaw/clawhub → add your folder → open a PR.

(GitHub account must be $\geq$ 1 week old.)

Security & Ethics

Why the stakes are high

The agent can touch everything

Local files ■ browser state ■ SaaS sessions ■ OAuth tokens ■ command execution.

A bug here is not a wrong answer — it is **data theft or remote code execution**.

- Feb 2026: **40,214** internet-exposed instances observed (SecurityScorecard); ~35% flagged vulnerable.
- Thousands susceptible to remote code execution in early scans.

Notable 2026 CVEs (case studies)

CVE	Type	Note
2026-32922	Privilege escalation	CVSS 9.9 ; <code>device.token.rotate</code>
2026-25253	Token leak	auto-connect to attacker <code>gatewayUrl</code>
2026-24763	Command injection	
2026-26322	SSRF	
2026-26329	Path traversal	local file reads
2026-30741	Prompt-injection RCE	via malicious content

“Claw Chain” (Cyera) chained four flaws → data theft + persistence.

Supply chain: the “ClawHavoc” incident

- Jan 2026: researchers (Koi) found **341 malicious skills** on ClawHub.
- Typosquatted names + fake “prerequisite” steps delivered the AMOS macOS stealer, reverse shells, credential exfiltration.
- Grew to **800+** flagged skills by mid-Feb 2026.

The lesson

Installing a skill = running someone else's code. Anyone with a week-old GitHub account could publish, with little review.

Hardening checklist (handout)

- Isolate: VM / container
- Least-privilege tokens + spend caps
- No broad network binding
- Strong authentication
- Vet every skill's source code
- Keep the agent updated
- Never run with production secrets present
- Log & review actions

Convenience and blast radius scale together.

Landscape & Wrap

OpenClaw vs. Gemini Spark

	OpenClaw	Gemini Spark
Hosting	Self-hosted (your box)	Google Cloud, always on
Ease	Needs setup/upkeep	Beginner-friendly
Models	Any (Claude/GPT/Gemini. . .)	Gemini only
Integrations	MCP → almost anything	Best inside Google Workspace
Sweet spot	Cross-boundary workflows	Tasks inside Google

Others: Anthropic Claude Cowork, OpenAI ChatGPT Agent, managed hosts (MyClaw).

How *you* use these skills

- **Automate pipelines:** scrape → clean → store → notify.
- **Monitoring agents:** watch a dataset/endpoint, alert on drift or anomalies.
- **Research assistant:** summarize papers & threads, draft reports.
- **Portfolio project:** a custom Skill + MCP server is a strong, demonstrable artifact.
- **Career:** “agent engineering” is emerging — understanding the *security tradeoffs* sets you apart.

Discuss:

1. Why might a university or bank *ban* self-hosted agents? Justified?
2. When is local-first safer than cloud — and when is it the opposite?
3. A useful ClawHub skill has 12 stars, no reviews. Install it? How decide?

Assignment (pick one):

- A **Build** a minimal Skill (SKILL.md + demo video). *Sandbox required.*
- B **Analyze** one CVE → 1-page threat model (vector, blast radius, fix).
- C **Compare** OpenClaw vs. Spark vs. a plain script for one DS task.

Key resources

Official: docs.openclaw.ai/start/getting-started ▪ github.com/openclaw/openclaw ▪ github.com/openclaw/clawhub ▪ Wikipedia: “OpenClaw”

Tutorials: DataCamp “Building Custom OpenClaw Skills” ▪ DEV “Zero to AI Assistant in 10 Minutes” ▪ VoltAgent [awesome-openclaw-skills](#)

News/context: Lex Fridman #491 (Steinberger) ▪ The New Stack “OpenClaw vs Gemini Spark”

Security: The Hacker News (four flaws) ▪ ARMO (CVE-2026-32922) ▪ Microsoft Security Blog (running it safely) ▪ Sangfor ▪ DigitalOcean

Many community blogs in this space are AI-generated — verify any exact command/number against the official docs or vendor security pages.



Thank you

Questions?

Lecture 15 — OpenClaw | Data Science with Python